

Part 1 - What is REST API?

The REST API is a REpresentational State Transfer that connects a distributed hypermedia system. Dr. Roy Fielding developed it in 2000 while doing his doctoral dissertation. The REST gives developers high level of flexibility, consistency, scalability and efficiency.

The REST API is based on the constraints and principles that promote simplicity, scalability, and statelessness in the design.

Six Guiding Principles of REST

Uniform Interface

The most important REST constraint. It includes:

- Resources are identified by URLs (e.g., /students/25).
- The server sends data in formats like JSON or XML, and the client uses that to understand or update the resource.
- Self-descriptive messages explain how to process the information (e.g., headers, status codes, content type).
- HATEOAS (Hypermedia As The Engine Of Application State) - optional in many modern APIs. Responses can include links that show the client what actions are possible next (like “edit,” “delete,” or “view more”).

REST provides a clear and consistent way for clients and servers to communicate. It uses standard HTTP methods like GET, POST, PUT, and DELETE, and it uses URLs (URIs) to point to the exact resources the client wants to access (IBM, n.d.).

Client–Server

- Client and server are separate — each has its own job.
- The client only needs the resource URL to make requests.
- The client cannot change the server’s internal logic — it just asks for data.
- The server only sends data back and does not modify the client.
- Each side can be updated independently without breaking the other.
- UI and data storage are separated, making apps easier to run on many devices.
- This design improves scalability, so the server can handle more users.
- The API contract must stay consistent, even if both sides evolve (IBM, n.d.).

Statelessness

- Every request the client sends must include all the information the server needs.
- The server cannot rely on any saved session or past information.
- Because of this, the client is responsible for keeping all session or state data, not the server (Gupta, 2025).

Cacheability

- Whenever possible, the client or server should be able to save (cache) responses.
- The server's response must clearly say whether the data can or cannot be cached.
- This helps the client load information faster and reduces repeated work for the server.
- A response must always state if it is cacheable or not — either directly or indirectly.
- If the response is cacheable, the client is allowed to reuse the saved data for similar requests for a certain amount of time (IBM, n.d.).

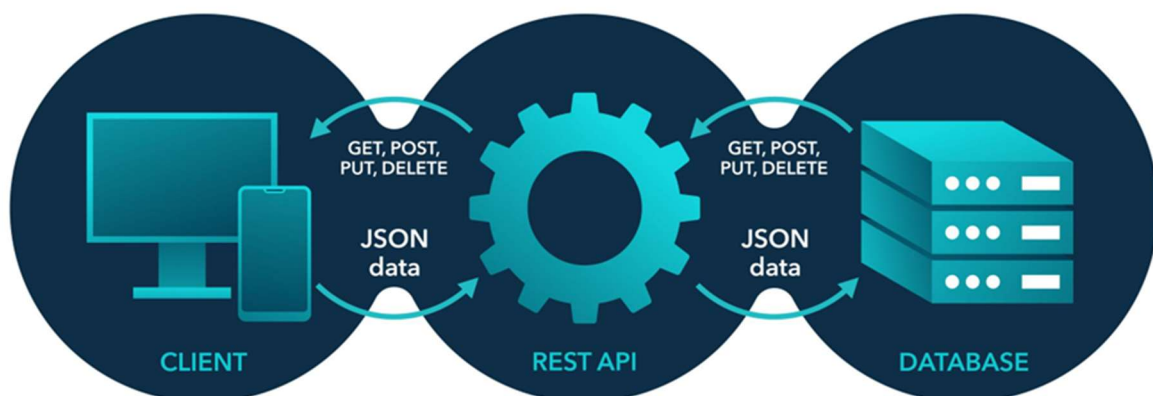
Layered System

- REST systems use multiple layers between the client and server.
- The client should not assume it connects directly to the server.
- Requests may pass through things like proxies, gateways, firewalls, or load balancers.
- The design must hide these layers so that neither the client nor server can tell if there is an intermediary.
- This makes the system easier to secure, scale, and maintain (IBM, n.d.).

Code on Demand (Optional)

- Normally, REST APIs send static data like JSON or HTML.
- But sometimes the server can also send executable code (like scripts or small programs).
- This downloaded code only runs when the client chooses to use it — on demand.
- It allows the server to provide extra features without the client needing to have them built in.
- This can make the client simpler because it doesn't need to implement everything itself.
- This is an optional REST feature and not commonly used today (Gupta, 2025).

How does requesting an API work?



The client sends requests (GET, POST, PUT, DELETE) to the REST API.

The REST API processes these requests and communicates with the database to read or update data.

The API then returns the results to the client in JSON format.

HTTP Methods

- GET request to retrieve a record
- POST request creates a new record
- PUT request updates a record
- PATCH updates only part of a resource
- DELETE request deletes one

Query String Parameters

- Added at the end of a URL after a question mark “?”
- Used to send small pieces of information to the server.
- Often used for filtering, searching, sorting, or identifying resources.
- Written as key=value pairs.
- Multiple parameters are separated using &.

Example: /products?category=shoes&color=blue

Request Body

- Data sent inside an HTTP request, not in the URL.
- Commonly used with POST, PUT, or PATCH methods.
- Used when creating or updating resources.
- Can contain larger or more complex data.
- Data is usually formatted as JSON, XML, or form data.

Example: JSON

```
{  
  "name": "Sneakers",  
  "color": "blue",  
  "price": 59.95  
}
```

Methods which I intend to use examples:

GET /users/{user_id}

- Purpose: Get one user profile
- QueryString: none
- Request Body: none

GET /planning?couple_id=25

- Purpose: Fetch our wedding planning.
- QueryString: couple_id
- Request Body: none

PATCH /planning/{couple_id}

- Purpose: Update specific fields (e.g., tick /untick checklist item)
- QueryString: none
- Request Body (JSON):

```
{  
  
  "florist": true  
  
}
```

POST /auth/register

- Purpose: Create a new user account
- QueryString: none
- Request Body (JSON):

```
{  
  "first_name": "Kimberly",  
  "last_name": "Parascandalo",  
  "email": "kim@gmail.com",  
  "password": "*****",  
}
```

DELETE /user/profile_image

- Purpose: Remove the profile image
- QueryString: none
- Request Body: none

PUT / our_wedding/{couple_id}

- Purpose: Update the wedding details for the couple
- QueryString: none
- Request Body (JSON):

```
{  
  
  "username1": "Kimmy",  
  "username2": "Joe",  
  "wedding_date": "2027-06-18",  
  "ceremony_venue": false ,  
  "reception_venue": true ,  
  "photographer": false ,  
  "guest_count": 150,  
  "budget": 20000  
}
```

Part 2 - What is OAuth 2.0

- OAuth 2.0 stands for “Open Authorisation”, which allows applications to access a user’s data from another service without sharing the user’s password.
- Instead of giving credentials directly, the user grants permission and the application receives a temporary access token to access the requested resources.
- It works through a system where a client application requests permission from an authorisation server, which then issues a token that can be used to access a protected resource (auth0, n.d.).

Benefits

- Improved security - Applications do not need to store or handle user passwords.
- Limited access control - Permissions can be restricted only to specific data or actions.
- Token access - Tokens can expire or be revoked, reducing long-term risk.
- Standardised framework - Widely supported by modern APIs and authentication systems.
- Separation of roles - Authentication and authorisation can be handled by trusted identity providers (Bello, 2023).

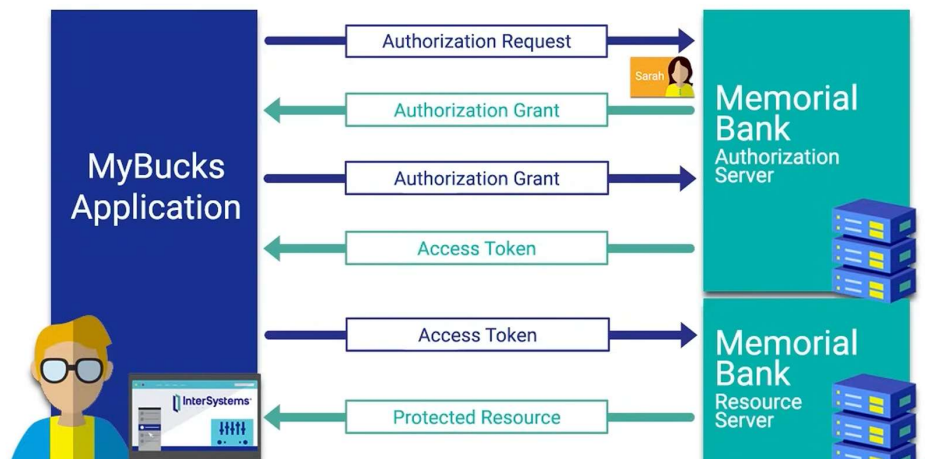
How OAuth 2.0 is Implemented

Typical implementation steps:

- The client application registers with an authorisation server and receives a Client ID and Client Secret.
- The application requests authorisation from the user.
- The authorisation server issues an access token.

- The client uses this token to send requests to the API or resource server.
- The resource server validates the token before granting access.

Workflow of OAuth 2.0



Scopes

- Scopes define the permissions that the application is requesting.
- They limit what the application can do with the user's data, for example: read the messages or write a post.
- The authorisation server checks scopes before issuing the token, ensuring that the application only receives the permissions approved by the user (auth0, n.d.).

Access Tokens

- An access token is a temporary credential used to access protected resources or APIs.
- The client application sends this token in API requests to prove it has permission.
- Tokens usually:
 - Have an expiration time
 - Are issued by the authorisation server
 - Can be revoked or refreshed if necessary.
- Access tokens allow applications to call APIs without exposing user credentials (Bello, 2023).

Client ID and Client Secret

- When an application registers with the authorisation server, it receives:

Client ID

- A public identifier for the application.
- Used to identify which application is requesting authorisation.

Client Secret

- A private key known only to the application and authorisation server.
- Used to authenticate the application when requesting tokens.
- These credentials are sent when requesting an access token from the OAuth token endpoint (auth0, n.d.).

How do I intend to implement this system within my own API?

I intend to use OAuth 2.0 in the login feature. When a user creates an account, their details are stored in the database. Then, when they log in, the system authenticates them and generates an access token. This token would then be used to access protected endpoints, such as their own wedding planning.

What is the OWASP API Security Top 10?

The Open Worldwide Application Security Project (OWASP) is an international non-profit organisation that focuses on improving the security of web applications and software. It provides free resources, documentation, tools, and guidelines that help developers and organisations understand and prevent common security vulnerabilities (Akamai, n.d.).

Overall, OWASP helps increase awareness of cybersecurity issues and encourages developers to build safer and more secure systems (Akamai, n.d.).

1. Broken Access Control

- This occurs when users can access data or perform actions that they are not authorised to do.
- For example, a normal user might be able to view or modify another user's account or access admin functions.
- It is considered the most serious web application risk because it can lead to data breaches or privilege escalation (van der Stock, et al., 2025).

2. Security Misconfiguration

- Security misconfiguration happens when systems, servers, or frameworks are incorrectly configured.
- Examples include leaving default passwords, exposing unnecessary services, or missing security headers.
- These mistakes can leave applications vulnerable to attackers (van der Stock, et al., 2025).

3. Software Supply Chain Failure

- This threat happens when an application uses third-party software components, such as libraries, frameworks, plugins, or packages that come from external developers.
- Many modern applications depend on these external components to save development time.

- If one of these components contains a security vulnerability or malicious code, attackers may be able to exploit it to access the system or its data (van der Stock, et al., 2025).

4. Cryptographic Failures

- Cryptographic failures occur when sensitive data is not properly protected using encryption.
- This can expose passwords, personal information, or financial data.
- Weak encryption methods or missing encryption can allow attackers to hijack or steal data (van der Stock, et al., 2025).

5. Injection

- Injection vulnerabilities happen when untrusted input is sent to an interpreter as part of a command or query.
- Common examples include SQL injection and Cross-Site Scripting (XSS).
- Attackers can use these vulnerabilities to execute malicious code or access sensitive data (van der Stock, et al., 2025).

6. Insecure Design

- Insecure design refers to weaknesses in the application's architecture or design.
- These issues occur when security is not considered during the development process.
- Examples include missing security controls or poor system design that allows attackers to exploit the application (van der Stock, et al., 2025).

7. Authentication Failures

- Authentication failures occur when systems do not properly verify user identities.
- Weak login systems may allow attackers to guess passwords, bypass authentication, or hijack sessions.
- This can lead to unauthorised access to accounts or sensitive data (van der Stock, et al., 2025).

8. Software or Data Integrity Failures

- This threat occurs when software updates, code, or data cannot be trusted because their integrity is not verified.
- For example, attackers might insert malicious code into software updates or modify data during transmission.
- Proper validation and integrity checks are required to prevent this (van der Stock, et al., 2025).

9. Security Logging and Alerting Failures

- This risk appears when systems fail to record security events or detect suspicious activity.
- Without proper logging and monitoring, attacks may go unnoticed for a long time.
- This makes it difficult to detect breaches and respond quickly (van der Stock, et al., 2025).

10. Mishandling of Exceptional Conditions

- This refers to poor handling of errors or unexpected conditions in an application.

- Examples include systems that crash, reveal sensitive information in error messages, or behave unpredictably during failures.
- These weaknesses can help attackers discover vulnerabilities or exploit system weaknesses (van der Stock, et al., 2025).

How do I intend to mitigate these threats?

Broken Access Control

- To stop users from accessing data or functions that they are not authorised to use. I will implement role-based access control (RBAC) in my API. For example, couples will only be able to view and manage their own wedding plans.

Authentication Failures

- To prevent the users from hijacking sessions or guessing passwords. I will try to make a verification code when users forget their passwords or suggest a strong password

Injection

- To prevent injection attacks, I will ensure that all user input is properly validated before it is processed by the system, so that attackers cannot write malicious code in the login form to bypass authentication. For example, ' OR '1'='1

References

Akamai, n.d. *What Is OWASP?*. [Online]

Available at: <https://www.akamai.com/glossary/what-is-owasp>

[Accessed 11 March 2026].

auth0, n.d. *What is OAuth 2.0?*. [Online]

Available at: <https://auth0.com/intro-to-iam/what-is-oauth-2>

[Accessed 7th March 2026].

Bello, G., 2023. *What is OAuth 2.0?*. [Online]

Available at: <https://blog.postman.com/what-is-oauth-2-0/>

[Accessed 7th March 2026].

Gupta, L., 2025. *What is REST?*. [Online]

Available at: <https://restfulapi.net/>

[Accessed 27 February 2026].

IBM, n.d. *What is REST API?*. [Online]

Available at: <https://www.ibm.com/think/topics/rest-apis>

[Accessed 27 February 2026].

van der Stock, A. et al., 2025. *The Ten Most Critical Web Application Security Risks*.

[Online]

Available at: <https://owasp.org/Top10/2025/>

[Accessed 11 March 2026].